
Daily Progress Report

Week 4_Day1: Timers & The Event Loop

Program	3-Month MERN Stack Internship Program
Intern Name	Mohsin Ali
Week	Week_4 (Day 1)
Date	13 July, 2026

Day 1: Timers & The Event Loop

1.1 Topics Studied

- Synchronous vs. Asynchronous JavaScript execution
- JavaScript Call Stack
- Browser Web APIs
- Callback Queue
- JavaScript Event Loop
- setTimeout() and clearTimeout()
- setInterval() and clearInterval()
- DOM manipulation using JavaScript
- Event handling and user interactions
- Time formatting and state management
- Building responsive and interactive web interfaces using vanilla JavaScript

1.2 Task Description

Developed a Digital Stopwatch application using pure vanilla JavaScript to understand the practical implementation of JavaScript timers and asynchronous execution. The application provides users with Start, Stop, and Reset controls to accurately measure elapsed time in **MM:SS:MS** (Minutes:Seconds:Milliseconds) format. The primary objective of this project was to gain hands-on experience with setInterval(), timer management, DOM manipulation, and event handling while ensuring the application remained reliable by preventing multiple timer instances when the Start button is clicked repeatedly.

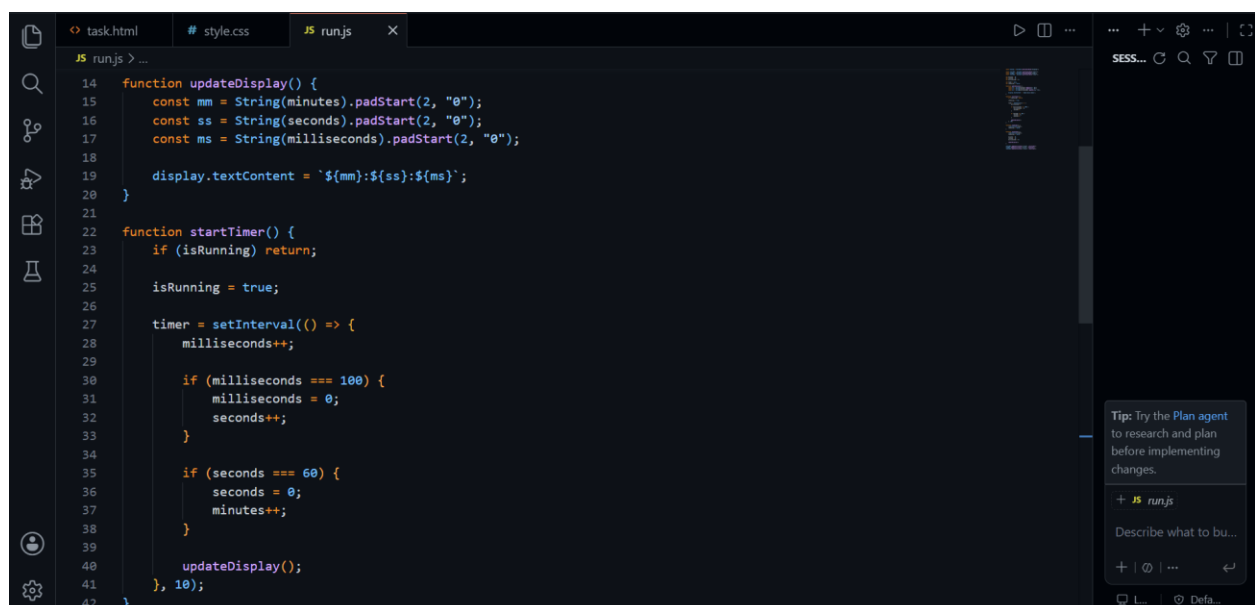
1.3 Implementation Details

- Designed a modern, responsive, and user-friendly stopwatch interface using HTML and CSS.
- Implemented a real-time digital stopwatch using JavaScript's `setInterval()` function.
- Formatted the elapsed time in **MM:SS** format using JavaScript string formatting techniques.
- Added **Start**, **Stop**, and **Reset** controls to manage stopwatch functionality.
- Prevented multiple timer instances by implementing state management logic, ensuring the stopwatch does not glitch when the Start button is clicked multiple times.
- Used `clearInterval()` to stop the running timer efficiently and avoid unnecessary resource usage.
- Implemented reset functionality to clear the timer and restore the stopwatch to its initial state.
- Updated the stopwatch display dynamically through DOM manipulation without requiring page refreshes.
- Organized the JavaScript code into modular functions to improve readability, maintainability, and scalability.
- Designed the interface to be fully responsive, ensuring smooth functionality across desktop, tablet, and mobile devices.

1.4 Outcome

The Digital Stopwatch application was successfully completed and fulfilled all the required functional requirements. The project provided practical experience with JavaScript's asynchronous behaviour, timers, the Event Loop, DOM manipulation, and event handling. It also strengthened my understanding of managing application state, preventing duplicate timer execution, and building responsive, interactive web applications using pure vanilla JavaScript while following clean coding practices.

Screenshots:



The screenshot shows a code editor with three tabs: task.html, style.css, and run.js. The style.css tab is active, displaying the following CSS code:

```
# style.css > .buttons
1  * {
2    margin: 0;
3    padding: 0;
4    box-sizing: border-box;
5  }
6
7  body {
8    font-family: 'Poppins', sans-serif;
9    height: 100vh;
10   display: flex;
11   justify-content: center;
12   align-items: center;
13   background: linear-gradient(135deg, #0f172a, #1e293b);
14   overflow: hidden;
15 }
16
17 .background-circle {
18   position: absolute;
19   border-radius: 50%;
20   filter: blur(90px);
21 }
22
23 .circle1 {
24   width: 350px;
25   height: 350px;
26   background: #38bdf8;
27   top: -120px;
28   left: -120px;
29   opacity: .4;
```

On the right side of the editor, there is a sidebar with a search bar and a list of files. A tip box at the bottom right suggests using the Plan agent for research and planning.

Screenshots:

